

Test Cases

Suite 1: createRootFolders Mutation

Suite 2: createFolder Mutation

Suite 3: updateFolder Mutation

Suite 4: moveFolder Mutation

Suite 5: moveFolders Mutation

Suite 6: deleteFolder Mutation

Test Cases

Suite 1: **createRootFolders** Mutation

Test Case: TC_CR_01 - Initial Tenant Bootstrap with Explicit Domain

Priority: High | **Type:** Functional

Pre-conditions:

- Tenant organization exists with **{{ORG_ID}}**.
- Valid authorization token **{{VALID_TOKEN}}** with organization administrator privileges.
- Tenant root folder partition for domain **cms** is uninitialized.

Step	Action / Step Description	Input Data	Expected Result
1	Execute createRootFolders mutation specifying explicit domain partition	rootFolderType: cms , Header: Authorization: Bearer {{VALID_TOKEN}}	HTTP 200 OK. Returns an array [Folder] containing at least one root folder object with non-null id , non-null name (e.g., "Root CMS"), and rootFolderTypeId: 1 .

Post-conditions:

- Tenant organization has root folder anchor initialized and registered for the **cms** partition.

Test Case: TC_CR_02 - Tenant Bootstrap with Default Parameter

(Omitted **rootFolderType**)

Priority: Medium | **Type:** Functional

Pre-conditions:

- Tenant organization exists with **{{ORG_ID}}**.
- Valid authorization token **{{VALID_TOKEN}}**.
- Default root partition (**watchlist**) is uninitialized.

Step	Action / Step Description	Input Data	Expected Result
1	Execute createRootFolders mutation without providing rootFolderType argument	None (default parameter takes effect), Header: Authorization: Bearer {{VALID_TOKEN}}	HTTP 200 OK. Schema default argument watchlist applies. Returns an array [Folder] containing initialized default root folder with valid id and rootFolderTypeId matching watchlist domain.

Post-conditions:

- Default **watchlist** root folder partition is initialized for the active organization.

Test Case: TC_CR_03 - Idempotent Re-invocation on Already Initialized

Tenant

Priority: High | **Type:** Functional

Pre-conditions:

- Tenant organization **{{ORG_ID}}** has previously initialized root folders for **rootFolderType: cms** with root ID **{{ROOT_CMS_ID}}**.
- Valid authorization token **{{VALID_TOKEN}}**.

Step	Action / Step Description	Input Data	Expected Result
1	Re-execute createRootFolders mutation for the existing domain partition	rootFolderType: cms , Header: Authorization: Bearer {{VALID_TOKEN}}	HTTP 200 OK. Operation executes idempotently: returns existing root folder array with id = "{{ROOT_CMS_ID}}" without creating duplicate root records.

Post-conditions:

- Root hierarchy remains stable and uncorrupted; no duplicate root folders created.

Test Case: TC_CR_04 - Unauthorized Execution under Invalid or Missing Tenant Token

Priority: High | **Type:** Functional

Pre-conditions:

- Invalid, expired, or malformed authorization token `{{INVALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>createRootFolders</code> mutation with invalid authentication credentials	<code>rootFolderType: cms</code> , Header: <code>Authorization: Bearer {{INVALID_TOKEN}}</code>	GraphQL / HTTP error response. Returns error code <code>UNAUTHENTICATED</code> or <code>FORBIDDEN</code> with authentication rejection message; <code>data.createRootFolders</code> is null.

Post-conditions:

- Request is rejected; no root folders are initialized or modified.
-

Suite 2: `createFolder` Mutation

Test Case: TC_CF_01 - Standard Child Folder Creation Under Root Node

Priority: High | **Type:** Functional

Pre-conditions:

- Valid session with `{{VALID_TOKEN}}` in organization `{{ORG_ID}}`.
- Initialized root folder with `{{ROOT_CMS_ID}}` in domain `cms`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>createFolder</code> mutation attaching	<code>name: "Raw Footage"</code> , <code>description:</code>	HTTP 200 OK. Returns <code>Folder</code> object with

Step	Action / Step Description	Input Data	Expected Result
	new folder to root node	"Inbound unedited media streams", parentId: "{{ROOT_CMS_ID}}", rootFolderType: cms, Header: Authorization: Bearer {{VALID_TOKEN}}	generated UUID id, name = "Raw Footage", and parentFolderId = "{{ROOT_CMS_ID}}".

Post-conditions:

- New folder node is created in the virtual directory tree linked directly to {{ROOT_CMS_ID}}.

Test Case: TC_CF_02 - Multi-Tier Hierarchy Creation (Deeply Nested Child Folder)

Priority: Medium | **Type:** Functional

Pre-conditions:

- Intermediate parent folder exists with {{PARENT_FOLDER_A_ID}} in domain cms.
- Valid session with {{VALID_TOKEN}}.

Step	Action / Step Description	Input Data	Expected Result
1	Execute createFolder mutation targeting intermediate parent folder	name: "Camera-Unit-01", description: "Daily ingest batch 1", parentId: "{{PARENT_FOLDER_A_ID}}", rootFolderType: cms, Header: Authorization: Bearer	HTTP 200 OK. Returns Folder object with generated UUID id, name = "Camera-Unit-01", and parentFolderId = "{{PARENT_FOLDER_A_ID}}".

Step	Action / Step Description	Input Data	Expected Result
		{{VALID_TOKE N}}	

Post-conditions:

- Multi-tier folder hierarchy is expanded; child folder is linked to {{PARENT_FOLDER_A_ID}}.

Test Case: TC_CF_03 - Sibling Name Collision Under Identical Parent

Priority: Medium | **Type:** Functional

Pre-conditions:

- Existing folder with name "Camera-Unit-01" already exists under parent {{PARENT_FOLDER_A_ID}}.
- Valid session with {{VALID_TOKEN}}.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>createFolder</code> mutation using identical name under same parent	name: "Camera-Unit-01", parentId: "{{PARENT_FOLDER_A_ID}}", rootFolderType: cms, Header: Authorization: Bearer {{VALID_TOKE N}}	System either permits duplicate sibling name with a newly generated unique <code>id</code> (as per DAG spec) or returns validation error <code>CONFLICT / BAD_USER_INPUT</code> if tenant sibling uniqueness is enforced.

Post-conditions:

- If permitted, both records exist with distinct UUIDs; if restricted, database state is unchanged.

Test Case: TC_CF_04 - Folder Creation with Non-Existent or Deleted

parentId

Priority: High | **Type:** Functional

Pre-conditions:

- Non-existent parent ID `{{NON_EXISTENT_ID}}`.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>createFolder</code> mutation referencing non-existent parent folder	<code>name: "Orphan Folder", parentId: "{{NON_EXISTENT_ID}}", rootFolderType: cms, Header: Authorization: Bearer {{VALID_TOKEN}}</code>	GraphQL error response. Returns error code <code>NOT_FOUND</code> or <code>BAD_USER_INPUT</code> indicating parent folder does not exist or foreign key constraint violation. <code>data.createFolder</code> is null.

Post-conditions:

- No orphan folder is created in the database.

Test Case: TC_CF_05 - Cross-Domain Partition Mismatch

Priority: High | **Type:** Functional

Pre-conditions:

- Root folder `{{ROOT_WATCHLIST_ID}}` belongs to partition `watchlist`.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>createFolder</code> mutation specifying <code>rootFolderType: cms</code> under <code>watchlist</code> root	<code>name: "Cross Partition Folder", parentId: "{{ROOT_WATCHLIST_ID}}", rootFolderType: cms, Header: Authorization: Bearer {{VALID_TOKEN}}</code>	GraphQL error response. Returns validation error code <code>BAD_USER_INPUT</code> indicating child domain partition must match parent domain partition.

Step	Action / Step Description	Input Data	Expected Result
			<code>data.createFolder</code> is null.

Post-conditions:

- Domain partition integrity is preserved; no cross-partition folder node is instantiated.

Test Case: TC_CF_06 - Input Boundary Validation (Empty Name, Whitespace Strings)

Priority: Medium | **Type:** Functional

Pre-conditions:

- Valid parent folder `{{PARENT_FOLDER_A_ID}}` exists.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>createFolder</code> mutation with whitespace-only name string	<code>name: " ",</code> <code>parentId: "{{PARENT_FOLDER_A_ID}}",</code> <code>rootFolderType: cms,</code> Header: <code>Authorization: Bearer {{VALID_TOKEN}}</code>	GraphQL validation error. Returns <code>BAD_USER_INPUT</code> indicating <code>name</code> must be a valid non-empty string with length <code>\$\ge 1\$</code> . <code>data.createFolder</code> is null.

Post-conditions:

- Invalid folder record is rejected; database state remains clean.
-

Suite 3: `updateFolder` Mutation

Test Case: TC_UF_01 - In-Place Renaming and Metadata Update

Priority: High | **Type:** Functional

Pre-conditions:

- Folder exists with ID `{{FOLDER_ID}}`, current name "Camera-Unit-01", and parent `{{PARENT_FOLDER_A_ID}}`.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>updateFolder</code> mutation with mutated name	id: <code>"{{FOLDER_ID}}"</code> , name: "Camera-Unit-01 [Priority Ingest]", Header: Authorization: Bearer <code>{{VALID_TOKEN}}</code>	HTTP 200 OK. Returns Folder object with id = <code>"{{FOLDER_ID}}"</code> , mutated name = "Camera-Unit-01 [Priority Ingest]", and unchanged parentFolderId = <code>"{{PARENT_FOLDER_A_ID}}"</code> .

Post-conditions:

- Folder name is updated in-place; parentage and tree topology remain unchanged.

Test Case: TC_UF_02 - Idempotent Update with Unchanged Name

Priority: Low | **Type:** Functional

Pre-conditions:

- Folder exists with ID `{{FOLDER_ID}}` and name "Camera-Unit-01".
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>updateFolder</code> mutation passing exact current name	id: <code>"{{FOLDER_ID}}"</code> , name: "Camera-Unit-01", Header: Authorization: Bearer <code>{{VALID_TOKEN}}</code>	HTTP 200 OK. Operation completes as a no-op; returns Folder object with id = <code>"{{FOLDER_ID}}"</code> and name = "Camera-Unit-01".

Post-conditions:

- Folder attributes remain identical; no unintended mutations occur.

Test Case: TC_UF_03 - Update Targeting Non-Existent Folder ID

Priority: High | **Type:** Functional

Pre-conditions:

- Folder ID `{{NON_EXISTENT_ID}}` does not exist in the tenant database.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>updateFolder</code> mutation targeting <code>{{NON_EXISTENT_ID}}</code>	id: <code>"{{NON_EXISTENT_ID}}"</code> , name: <code>"Ghost Folder"</code> , Header: <code>Authorization: Bearer {{VALID_TOKEN}}</code>	GraphQL error response. Returns error code <code>NOT_FOUND</code> indicating folder does not exist. <code>data.updateFolder</code> is null.

Post-conditions:

- No records are updated or created.
-

Test Case: TC_UF_04 - Rejection of Structural Mutation (Mutating Parentage via Update)

Priority: Medium | **Type:** Functional

Pre-conditions:

- Folder exists with ID `{{FOLDER_ID}}` under `{{PARENT_FOLDER_A_ID}}`.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>updateFolder</code> attempting to pass <code>parentId</code> in input payload	id: <code>"{{FOLDER_ID}}"</code> , name: <code>"Renamed"</code> , <code>parentId</code> : <code>"{{PARENT_FOLDER_A_ID}}"</code>	GraphQL schema validation error. Unknown argument or input field <code>parentId</code> on input type

Step	Action / Step Description	Input Data	Expected Result
		DER_B_ID}}", Header: Authorization: Bearer {{VALID_TOKEN}}	UpdateFolder. Operation is rejected before execution.

Post-conditions:

- Directory tree topology remains unaltered; structural moves remain restricted to moveFolder/moveFolders.

Test Case: TC_UF_05 - UTF-8 Character Handling (Special Characters, Unicode, Emojis)

Priority: Low | **Type:** Functional

Pre-conditions:

- Folder exists with ID {{FOLDER_ID}}.
- Valid session with {{VALID_TOKEN}}.

Step	Action / Step Description	Input Data	Expected Result
1	Execute updateFolder mutation with multilingual Unicode characters, symbols, and emojis	id: "{{FOLDER_ID}}", name: "📁 Ingest [2026] / Équipe & テスト #42", Header: Authorization: Bearer {{VALID_TOKEN}}	HTTP 200 OK. Successfully stores and returns UTF-8 string name = "📁 Ingest [2026] / Équipe & テスト #42".

Post-conditions:

- Folder name accurately reflects Unicode formatting without encoding corruption.
-

Suite 4: **moveFolder** Mutation

Test Case: TC_MF_01 - Valid Single-Folder Relocation Between Distinct Parents

Priority: High | **Type:** Functional

Pre-conditions:

- Folder `{{SUBFOLDER_ID}}` currently resides under parent `{{PARENT_FOLDER_A_ID}}`.
- Destination parent `{{PARENT_FOLDER_B_ID}}` exists in the same domain.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute moveFolder mutation specifying valid source and target parent IDs	<code>folderId:</code> <code>"{{SUBFOLDER_ID}}",</code> <code>fromFolderId:</code> <code>"{{PARENT_FOLDER_A_ID}}",</code> <code>toFolderId:</code> <code>"{{PARENT_FOLDER_B_ID}}",</code> Header: Authorization: Bearer <code>{{VALID_TOKEN}}</code>	HTTP 200 OK. Returns Folder object with <code>id =</code> <code>"{{SUBFOLDER_ID}}"</code> and updated <code>parentFolderId =</code> <code>"{{PARENT_FOLDER_B_ID}}"</code> .

Post-conditions:

- `{{SUBFOLDER_ID}}` is detached from `{{PARENT_FOLDER_A_ID}}` and reparented under `{{PARENT_FOLDER_B_ID}}`.

Test Case: TC_MF_02 - Optimistic Concurrency Lock Collision

Priority: High | **Type:** Functional

Pre-conditions:

- Live database state: `{{SUBFOLDER_ID}}` is currently located under `{{PARENT_FOLDER_B_ID}}`.
- Stale client attempts move providing outdated source parent `{{PARENT_FOLDER_A_ID}}`.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>moveFolder</code> mutation with mismatched <code>fromFolderId</code>	<code>folderId: "{{SUBFOLDER_ID}}",</code> <code>fromFolderId: "{{PARENT_FOLDER_A_ID}}",</code> <code>toFolderId: "{{PARENT_FOLDER_C_ID}}",</code> Header: Authorization: Bearer <code>{{VALID_TOKEN}}</code>	GraphQL error response. Returns precondition failure or conflict error (<code>PRECONDITION_FAILED</code> , <code>CONFLICT</code> , or <code>BAD_USER_INPUT</code>) stating that <code>fromFolderId</code> does not match live parent. <code>data.moveFolder</code> is null.

Post-conditions:

- Concurrency safeguard protects data integrity; folder remains under its live parent `{{PARENT_FOLDER_B_ID}}`.

Test Case: TC_MF_03 - Self-Relocation Rejection (folderId == toFolderId)

Priority: Medium | **Type:** Functional

Pre-conditions:

- Folder exists with ID `{{FOLDER_ID}}` under parent `{{PARENT_FOLDER_A_ID}}`.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>moveFolder</code> mutation setting destination parent to the folder itself	<code>folderId: "{{FOLDER_ID}}",</code> <code>fromFolderId: "{{PARENT_FOLDER_A_ID}}",</code> <code>toFolderId: "{{FOLDER_ID}}",</code> Header:	GraphQL error response. Returns validation error code <code>BAD_USER_INPUT</code> indicating a folder cannot be its own parent.

Step	Action / Step Description	Input Data	Expected Result
		Authorization: Bearer {{VALID_TOKEN}}	data.moveFolder is null.

Post-conditions:

- Hierarchy topology is untouched; self-referencing cycle is prevented.

Test Case: TC_MF_04 - No-Op Relocation (fromFolderId == toFolderId)

Priority: Low | **Type:** Functional

Pre-conditions:

- Folder {{SUBFOLDER_ID}} currently resides under parent {{PARENT_FOLDER_A_ID}}.
- Valid session with {{VALID_TOKEN}}.

Step	Action / Step Description	Input Data	Expected Result
1	Execute moveFolder mutation where source parent equals destination parent	folderId: "{{SUBFOLDER_ID}}", fromFolderId: "{{PARENT_FOLDER_A_ID}}", toFolderId: "{{PARENT_FOLDER_A_ID}}", Header: Authorization: Bearer {{VALID_TOKEN}}	HTTP 200 OK. Operation completes as a no-op; returns Folder with unchanged parentFolderId = "{{PARENT_FOLDER_A_ID}}".

Post-conditions:

- Folder remains under {{PARENT_FOLDER_A_ID}} without state mutation.
-

Test Case: TC_MF_05 - Hierarchy Cycle Prevention (Moving Ancestor into Own Descendant)

Priority: High | **Type:** Functional

Pre-conditions:

- `{{PARENT_FOLDER_A_ID}}` is the direct ancestor of `{{SUBFOLDER_ID}}`.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>moveFolder</code> attempting to move ancestor <code>{{PARENT_FOLDER_A_ID}}</code> into child <code>{{SUBFOLDER_ID}}</code>	<code>folderId: "{{PARENT_FOLDER_A_ID}}", fromFolderId: "{{ROOT_CMS_ID}}", toFolderId: "{{SUBFOLDER_ID}}", Header: Authorization: Bearer {{VALID_TOKEN}}</code>	GraphQL error response. Cycle detection triggers error code <code>BAD_USER_INPUT</code> or <code>CONFLICT</code> indicating circular reference detected in directory graph. <code>data.moveFolder</code> is null.

Post-conditions:

- Tree acyclicity is preserved; ancestor folder remains under `{{ROOT_CMS_ID}}`.

Test Case: TC_MF_06 - Cross-Domain Migration Rejection

Priority: High | **Type:** Functional

Pre-conditions:

- Source folder `{{SUBFOLDER_ID}}` belongs to `cms` domain tree.
- Target parent `{{PARENT_WATCHLIST_ID}}` belongs to `watchlist` domain tree.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>moveFolder</code>	<code>folderId: "{{SUBFOLDER_ID}}</code>	GraphQL error response. Returns

Step	Action / Step Description	Input Data	Expected Result
	attempting cross-domain reparenting	ID}}", fromFolderId: "{{PARENT_CMS_ID}}", toFolderId: "{{PARENT_WAT_CHLIST_ID}}", Header: Authorization: Bearer {{VALID_TOKEN}}	validation error code BAD_USER_INP UT or domain boundary violation. data.moveFolder is null.

Post-conditions:

- Domain boundary isolation is preserved; cross-partition move is aborted.

Suite 5: moveFolders Mutation

Test Case: TC_BF_01 - Homogeneous Bulk Migration of Multiple Valid Folders

Priority: High | **Type:** Functional

Pre-conditions:

- Folders {{FOLDER_1_ID}} and {{FOLDER_2_ID}} exist under source parent in domain cms.
- Target parent {{PARENT_FOLDER_B_ID}} exists in domain cms.
- Valid session with {{VALID_TOKEN}}.

Step	Action / Step Description	Input Data	Expected Result
1	Execute moveFolders mutation with a list of valid folder IDs	folderIds: ["{{FOLDER_1_ID}}", "{{FOLDER_2_ID}}"], newParentFolderId: {{PARENT_FOLDER_B_ID}}	HTTP 200 OK. Returns MoveFoldersPayload with newParentFolderId = {{PARENT_FOLDER_B_ID}}

Step	Action / Step Description	Input Data	Expected Result
		"{{PARENT_FOL DER_B_ID}}", rootFolderType: cms, Header: Authorization: Bearer {{VALID_TOKE N}}	DER_B_ID}}", validFolderIds = ["{{FOLDER_1_I D}}", "{{FOLDER_2_I D}}"], invalidFolderIds = [], and success message.

Post-conditions:

- Both folders are reparented under {{PARENT_FOLDER_B_ID}}.

Test Case: TC_BF_02 - Partial-Success Migration with Mixed Valid and Invalid Folder IDs

Priority: High | **Type:** Functional

Pre-conditions:

- {{FOLDER_1_ID}} is a valid existing folder.
- {{NON_EXISTENT_ID}} is an invalid or deleted ID.
- Target parent {{PARENT_FOLDER_B_ID}} exists.
- Valid session with {{VALID_TOKEN}}.

Step	Action / Step Description	Input Data	Expected Result
1	Execute moveFolders mutation passing mixed batch of valid and invalid IDs	folderIds: ["{{FOLDER_1_I D}}", "{{NON_EXISTE NT_ID}}"], newParentFolderId : "{{PARENT_FOL DER_B_ID}}", rootFolderType: cms, Header: Authorization: Bearer	HTTP 200 OK. Partial-success semantics: Returns MoveFoldersPaylo ad containing validFolderIds = ["{{FOLDER_1_I D}}"], invalidFolderIds = ["{{NON_EXIST ENT_ID}}"], and partial-success message.

Step	Action / Step Description	Input Data	Expected Result
		{{VALID_TOKEN}}	

Post-conditions:

- {{FOLDER_1_ID}} is successfully reparented to {{PARENT_FOLDER_B_ID}}; invalid ID is reported without failing valid mutations.

Test Case: TC_BF_03 - Empty Batch Relocation (folderIds: [])

Priority: Medium | **Type:** Functional

Pre-conditions:

- Target parent {{PARENT_FOLDER_B_ID}} exists.
- Valid session with {{VALID_TOKEN}}.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>moveFolders</code> mutation with empty <code>folderIds</code> array	<code>folderIds: [],</code> <code>newParentFolderId</code> <code>:</code> <code>"{{PARENT_FOLDER_B_ID}}",</code> <code>rootFolderType:</code> <code>cms,</code> Header: Authorization: Bearer <code>{{VALID_TOKEN}}</code>	HTTP 200 OK or schema validation error. Returns <code>MoveFoldersPayload</code> with empty <code>validFolderIds = [],</code> empty <code>invalidFolderIds = [],</code> or <code>BAD_USER_INPUT</code> indicating non-empty array required.

Post-conditions:

- System state remains completely unchanged.

Test Case: TC_BF_04 - Destination Parent Contained in Batch Selection

Priority: Medium | **Type:** Functional

Pre-conditions:

- Valid folder `{{FOLDER_1_ID}}` and destination parent `{{PARENT_FOLDER_B_ID}}` exist.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>moveFolders</code> mutation where destination parent is included in <code>folderIds</code>	<code>folderIds:</code> [" <code>{{FOLDER_1_ID}}</code> ", " <code>{{PARENT_FOLDER_B_ID}}</code> "], <code>newParentFolderId</code> : " <code>{{PARENT_FOLDER_B_ID}}</code> ", <code>rootFolderType:</code> <code>cms</code> , <code>Header:</code> <code>Authorization:</code> <code>Bearer</code> <code>{{VALID_TOKEN}}</code>	HTTP 200 OK. Returns <code>MoveFoldersPayload</code> where <code>{{FOLDER_1_ID}}</code> is listed in <code>validFolderIds</code> and <code>{{PARENT_FOLDER_B_ID}}</code> is flagged in <code>invalidFolderIds</code> (cannot move folder into itself).

Post-conditions:

- `{{FOLDER_1_ID}}` is reparented; target parent `{{PARENT_FOLDER_B_ID}}` remains uncorrupted.

Test Case: TC_BF_05 - Bulk Volume Threshold / Large Batch Handling

Priority: Medium | **Type:** Functional

Pre-conditions:

- Batch containing 50+ folder IDs (exceeding recommended batch partitioning threshold).
- Target parent `{{PARENT_FOLDER_B_ID}}` exists.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>moveFolders</code> mutation with	<code>folderIds:</code> [50+ IDs], <code>newParentFolderId</code> :	System either processes batch within SLA returning

Step	Action / Step Description	Input Data	Expected Result
	large batch array (50+ items)	"{{PARENT_FOLDER_B_ID}}", rootFolderType: cms, Header: Authorization: Bearer {{VALID_TOKEN}}	validFolderIds and invalidFolderIds, or rejects request with BAD_USER_INPUT / PAYLOAD_TOO_LARGE enforcing batch size constraints.

Post-conditions:

- Gateway and database stability preserved; state matches returned payload.

Suite 6: deleteFolder Mutation

Test Case: TC_DF_01 - Standard Deletion of Empty Leaf Folder with Valid orderIndex

Priority: High | **Type:** Functional

Pre-conditions:

- Empty leaf folder exists with ID {{FOLDER_ID}} and orderIndex: 1 under parent {{PARENT_FOLDER_A_ID}}.
- Folder contains 0 child subfolders and 0 filed assets.
- Valid session with {{VALID_TOKEN}}.

Step	Action / Step Description	Input Data	Expected Result
1	Execute deleteFolder mutation on empty leaf folder	id: "{{FOLDER_ID}}", orderIndex: 1, Header: Authorization: Bearer {{VALID_TOKEN}}	HTTP 200 OK. Returns DeletePayload with confirmation message confirming successful deletion.

Post-conditions:

- `{{FOLDER_ID}}` is permanently removed from directory DAG; parent's child folder count decrements.

Test Case: TC_DF_02 - Deletion Prevention of Non-Empty Folder with Child Subfolders

Priority: High | **Type:** Functional

Pre-conditions:

- Parent folder `{{PARENT_FOLDER_A_ID}}` contains active child subfolder `{{SUBFOLDER_ID}}`.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>deleteFolder</code> mutation directly on parent folder without pruning child first	id: <code>"{{PARENT_FOLDER_A_ID}}"</code> , orderIndex: 1, Header: Authorization: <code>Bearer {{VALID_TOKEN}}</code>	GraphQL error response. Returns error code <code>FAILED_PRECONDITION</code> or <code>CONFLICT</code> indicating folder cannot be deleted because it contains child folders. <code>data.deleteFolder</code> is null.

Post-conditions:

- Parent folder and child subfolders remain intact; dangling child references are prevented.

Test Case: TC_DF_03 - Deletion of Folder with Filed Media Assets / TDOs

Priority: Medium | **Type:** Functional

Pre-conditions:

- Folder `{{FOLDER_ID}}` has filed Temporal Data Objects (TDOs).
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute deleteFolder mutation on folder containing filed assets	id: "{{FOLDER_ID}}" , orderIndex: 1 , Header: Authorization: Bearer {{VALID_TOKEN}}	HTTP 200 OK (folder association removed while TDOs remain intact in platform per multi-tag filing architecture) OR error code PRECONDITION_FAILED if unfiling is required first.

Post-conditions:

- Filed media assets (TDOs) are preserved in the platform; virtual association with folder is cleanly detached.

Test Case: TC_DF_04 - Stale or Mismatched orderIndex Deletion

Handling

Priority: Medium | **Type:** Functional

Pre-conditions:

- Sibling folders exist with live rebalanced indexes; target folder live index is **2**.
- Client provides stale **orderIndex: 5**.
- Valid session with **{{VALID_TOKEN}}**.

Step	Action / Step Description	Input Data	Expected Result
1	Execute deleteFolder mutation with mismatched orderIndex	id: "{{FOLDER_ID}}" , orderIndex: 5 , Header: Authorization: Bearer {{VALID_TOKEN}}	System either auto-reconciles live order index and deletes node with HTTP 200 OK, or returns validation error BAD_USER_INPUT / PRECONDITION

Step	Action / Step Description	Input Data	Expected Result
			<u>FAILED</u> indicating index mismatch.

Post-conditions:

- Sibling indexing integrity is maintained; remaining siblings rebalanced.

Test Case: TC_DF_05 - Deletion Prevention on Protected Root Partition

Nodes

Priority: High | **Type:** Functional

Pre-conditions:

- Root folder initialized by `createRootFolders` with ID `{{ROOT_CMS_ID}}`.
- Valid session with `{{VALID_TOKEN}}`.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>deleteFolder</code> attempting to delete root partition node	id: <code>"{{ROOT_CMS_ID}}"</code> , orderIndex: 0, Header: Authorization: Bearer <code>{{VALID_TOKEN}}</code>	GraphQL error response. Returns error code <code>FORBIDDEN</code> or <code>BAD_USER_INPUT</code> indicating root partition folders are immutable anchor nodes and cannot be deleted. <code>data.deleteFolder</code> is null.

Post-conditions:

- Root partition anchor remains intact and functional for the organization.

Test Case: TC_DF_06 - Double Deletion / Idempotent Deletion on

Non-Existent Folder

Priority: Medium | **Type:** Functional

Pre-conditions:

- Folder `{{NON_EXISTENT_ID}}` does not exist or has already been deleted.

- Valid session with {{VALID_TOKEN}}.

Step	Action / Step Description	Input Data	Expected Result
1	Execute <code>deleteFolder</code> mutation targeting non-existent folder	id: " <code>{{NON_EXISTENT_ID}}</code> ", orderIndex: 1, Header: Authorization: Bearer <code>{{VALID_TOKEN}}</code>	GraphQL error response. Returns error code <code>NOT_FOUND</code> indicating target folder does not exist. <code>data.deleteFolder</code> is null.

Post-conditions:

- System state is unchanged; no foreign key or database anomalies occur.